

Thiết kế chi tiết

Hồ sơ thiết kế chi tiết (Design Dossier)

Tài liệu này kể **toàn bộ** những gì nằm trong code đang chạy: từng file — nhiệm vụ, từng quyết định thiết kế kèm lý do và nguồn chính thống, và bằng chứng đã kiểm thật. Đây là tầng sâu nhất của kiến trúc tài liệu ba tầng của repo.

0. Triết lý: tài liệu ba tầng

Một repo nghiêm túc không nhồi mọi giải thích vào comment (comment dày là nhiều và mục ruỗng khi code đổi — vi phạm clean code). Thay vào đó:

Tầng	Ở đâu	Trả lời	Cho ai
1. Code sạch	thân file	CÁI GÌ đang làm	người sửa code
2. Header chuẩn 5 mục	đầu mỗi file	VÌ SAO (tóm tắt) + nguồn + cách chạy	người đọc một file
3. Hồ sơ này	docs/THIET_KE_CHI	TOÀN BỘ : quyết định, nguồn, bày, bằng chứng	người chấm, người kế thừa

Hai nguyên tắc xuyên suốt mọi file: **(a)** mọi quyết định kỹ thuật phải có nguồn chính thống (tài liệu OpenSSL/nginx/liboqs/CMake/man-page hoặc repo gốc trên GitHub) — không “nghe nói”; **(b)** mọi cơ chế phải được **chạy thử thật** trước khi tin — kết quả ghi ở mục 9 (Nhật ký bằng chứng).

1. Nhóm cấu hình

scripts/versions.env

Nhiệm vụ: một chỗ duy nhất ghim phiên bản + đường dẫn cho toàn dự án. **Quyết định & nguồn:** - Ghim **tag** thay vì branch (openssl-3.6.2, 0.14.0, 0.10.0, release-1.30.2): tag bất biến → hai máy clone hai ngày khác nhau vẫn cùng một mã nguồn — đúng chữ “reproducible” để §7.3. Đổi phiên bản = sửa một dòng, xóa nguồn cũ, FORCE=1. - Mọi prefix gom dưới ~/pqc (openssl, liboqs-ref, liboqs-neon, nginx, src): không đụng /usr → không bao giờ phá OpenSSL hệ thống, gỡ sạch = rm -rf ~/pqc. - Hai cây liboqs đặt tên theo *bản chất* (liboqs-ref / liboqs-neon) chứ không theo cờ build — người đọc CSV hiểu ngay variant. - JOBS mặc định nproc — Pi 4 RAM 4GB vẫn an toàn với liboqs/nginx; OpenSSL nặng nhất cũng qua được (đã chạy thật trên LOQ). **Kiểm:** bash -n sạch; mọi script khác source nó làm nguồn sự thật duy nhất.

scripts/setenv.sh

Nhiệm vụ: kích hoạt môi trường (PATH, LD_LIBRARY_PATH, PKG_CONFIG_PATH) trở về OpenSSL của ta. **Quyết định & nguồn:** - **Source, không chạy** — biến môi trường đặt trong tiến trình con biến mất theo nó; vì vậy file không có shebang-thực-thì tư duy, README nhắc dậm. - **Không rpath** toàn dự án, tìm thư viện lúc chạy qua LD_LIBRARY_PATH — theo đúng quy ước demos chính chủ OpenSSL

(repo openssl/openssl, thư mục demos/ hướng dẫn chạy với LD_LIBRARY_PATH); binary giữ tính di dời, cơ chế tường minh. - Tự dò lib/ vs lib64/ — OpenSSL chọn libdir theo nền tảng (x86_64 Ubuntu ra lib64). **Kiểm sống:** sau source, which openssl → ~/pqc/openssl/bin; chuỗi sự cố “openssl hệ thống + libcrypto của ta thiếu openssl.cnf” trong sandbox (mục 9, dòng 2026-06-11/N5) minh họa vì sao LD_LIBRARY_PATH phải đi *cặp* với binary đúng.

.gitignore

Quyết định: chặn build/, nguồn bên thứ ba, và **khóa** (*.key, *.key.pem — vá thêm vì cert của ta tên *.key.pem, mẫu *.key không khớp); **cố ý không chặn** data/ và analysis_out/ vì raw CSV + plots là deliverable §11.4 (đã có lúc ignore nhầm analysis_out — phát hiện khi đối chiếu đề, sửa kèm ghi chú ngay trong file).

2. Nhóm dựng môi trường

scripts/install_prereqs.sh

Nhiệm vụ: cài đủ — và chỉ đủ — gói cho toàn dự án, một lệnh. **Quyết định & nguồn:** ba đợt gói ánh xạ thẳng vào đề (§8.2 toolchain, §7.4+8.2 công cụ đo, §16 phân tích); linux-tools-\$ (uname -r) cài kèm -generic vì kernel cloud/Pi mang hậu tố riêng — perf phải khớp kernel đang chạy; **cố ý loại** libssl-dev (trộn header hệ thống với OpenSSL tự build là đúng lớp bug dự án này chống), Docker (cài theo docs.docker.com), valgrind/astyle (extras của liboqs, ngoài phạm vi). Hộp thoại tshark: chọn Yes + usermod -aG wireshark + đăng nhập lại (nhóm nạp lúc login). **Kiểm:** toàn bộ tên gói tra apt-cache Ubuntu 24.04 trước khi viết.

scripts/build_openssl.sh

Nhiệm vụ: dựng OpenSSL 3.6.2 PQC-native — *nền móng của mọi thứ*. **Quyết định & nguồn:** - git clone --depth 1 --branch <tag> từ github.com/openssl/openssl — tải đúng bản ghim, nông cho nhẹ. - Cấu hình --prefix=~/.pqc/openssl + **make install_sw install_ssldirs:** install_sw gọn (bỏ man pages), install_ssldirs tạo openssl.cnf — thiếu nó là req chết (đã *tự dẫm* trong sandbox khi đi tắt: mục 9/N5 — bằng chứng sống cho quyết định này). - Test suite đầy đủ chạy **ít nhất một lần** mỗi máy (bằng chứng đúng đắn); SKIP_TESTS=1 cho các lần sau/Pi/CI — quy ước giống oqs-demos. - Idempotent + FORCE=1; ghi docs/openssl.commit làm bằng chứng tái lập. **Kiểm sống:** bạn đã build thành công trên LOQ; sandbox build lại từ nguồn để dựng nginx (mục 9/N4).

scripts/build_liboqs.sh [ref|opt]

Nhiệm vụ: dựng **hai cây** liboqs cùng phiên bản — biến độc lập duy nhất là tối ưu vi kiến trúc → thí nghiệm RQ2. **Quyết định & nguồn:** - ref: C thuần, tắt tường minh SIMD (OQS_OPT_TARGET=generic); opt: -march=native/auto-dò NEON-AVX2 — đúng cách liboqs định nghĩa build tối ưu (README + CMake options của repo open-quantum-safe/liboqs, bản 0.14.0). - DIST_BUILD=OFF cho bản opt để CPU-extensions được dùng tối đa trên máy đo (tài liệu CONFIGURE.md của liboqs). - Khối ARM: bật/tắt NEON tường minh để hai cây *thật sự* khác nhau đúng một biến. - Giữ build-ref//build-opt/ (chứa tests/speed_*) — runner WP3 cần. **Kiểm sống:** số thật trên LOQ: ML-KEM-768 ref 30.985μs vs opt 10.705μs keygen (×2.89) — cycle cross-check khớp; banner hai cây in đúng “SSE SSE2”/AES:OpenSSL vs “AVX2.../AES:NI” — bằng chứng hai cây khác đúng chỗ cần khác (mục 9/N1).

scripts/fetch_pqclean.sh [clean|avx2|aarch64]

Nhiệm vụ: lấy PQClean và build .a **per-algorithm** — nguồn duy nhất cho code-size từng thuật toán (libcrypto/liboqs trộn mọi thứ, size không tách được). **Quyết định & nguồn:** - **Ghim theo commit** chứ không tag: tag mới nhất của PQClean là round2/round3 cổ — ghim tag là đo bản tiền-chuẩn hóa, sai mục tiêu. - Ghi nhận chính thức: PQClean **archive read-only 07/2026**, kế nhiệm PQ Code

Package (README chính chủ PQClean) — đưa vào Literature, dùng PQClean vẫn đúng vì FIPS-final code đã có trong commit ghim. - **Guard chéo kiến trúc:** aarch64 trên máy x86 bị chặn với thông điệp hướng dẫn — vì các `.S/___asm_NTT` viết tay không cross được bằng đường này. **Kiểm sống:** chính guard này sinh ra từ một lần **fail thật** trong sandbox (mục 9/N2) — cross-compile aarch64 trên x86 vớ ở `___asm_NTT.o`; fail được giữ lại làm thiết kế.

`scripts/build_oqsprovider.sh [ref|opt]`

Nhiệm vụ: lắp provider OQS vào OpenSSL của ta (đường tích hợp *kế nhiệm* fork OQS-OpenSSL 1.1.1 mà đề §7.2 nhắc — fork đã ngừng, README chính chủ `open-quantum-safe/openssl` trở sang `oqs-provider`). **Quyết định & nguồn:** truyền `OPENSSL_ROOT_DIR + liboqs_DIR` đúng nguyên văn README `oqs-provider 0.10.0`; activation qua `openssl.cnf` đúng tài liệu `provider(7)`. Vai trò trong đồ án: bằng chứng tích hợp + mở thuật toán ngoài chuẩn; **số đo chính vẫn từ ML-KEM/ML-DSA native** của OpenSSL 3.5+ (một lớp ít hơn, sạch hơn).

`scripts/cross_build_liboqs.sh + cmake/aarch64-toolchain.cmake`

Nhiệm vụ: bằng chứng §7.3 “separate cross-compile scripts (aarch64-linux-gnu toolchain)”. **Quyết định & nguồn:** phương pháp đúng wiki `liboqs` (“Cross-compiling on Linux for ARM”): `toolchain` file + `OQS_USE_OPENSSL=OFF` (host x86 không có OpenSSL ARM để link). `Toolchain` file soi gương ví dụ `liboqs` ship sẵn (`.CMake/toolchain_rasppi.cmake`) — bản của họ là `armhf/gcc-8 32-bit`, bản ta là `aarch64` hiện đại; bộ từ `FIND_ROOT_PATH_MODE` (thêm `PACKAGE` cho khớp upstream từng-dòng). **Ranh giới khác đá:** sản phẩm `cross` là bằng chứng *năng lực*, không bao giờ là nguồn số đo — số ARM chỉ đến từ máy ARM thật. **Kiểm sống:** `cross-configure` thật trong sandbox nhận GNU 13.3.0 `aarch64`, “Configuring done”, 0 cảnh báo cờ thừa.

`scripts/docker_buildx.sh + docker/Dockerfile.x86_64`

Nhiệm vụ: đóng hộp tái lập (§7.3 “Dockerfile cho x86 64, docker buildx multiarch”). **Quyết định & nguồn:** `Dockerfile` **tái dùng chính script của repo** (một nguồn sự thật — script đổi, image đổi theo); `verify_env.sh` cuối image = image tự-fail khi hỏng; `buildx` theo docs.docker.com/build/multi-platform, `binfmt` một lần; caveat `--load` với manifest đa nền tảng ghi sẵn. **Định luật:** không bao giờ đo trong QEMU — số của trình giả lập.

`scripts/build_nginx.sh + scripts/nginx_bench.conf`

Nhiệm vụ: dựng `nginx 1.30.2` **link động với OpenSSL 3.6.2 của ta** — vì `nginx` là “người thuê nhà” của `OpenSSL`: `apt-nginx link 3.0` hệ thống → không `ML-KEM` → vô dụng cho WP4. **Quyết định & nguồn:** - Mirror chính thức `github.com/nginx/nginx`, tag ghim, `auto/configure` (cách build từ git tree). - `---without-http_rewrite_module --without-http_gzip_module`: không kéo `PCRE/zlib-dev` (đúng triết lý prereqs tối thiểu). **Hệ quả đã dựng thật:** mất directive `return` → template trả file tĩnh (mục 9/N4). - Link **động** `--with-cc-opt/-ld-opt` thay vì `--with-openssl=src` (static): tái dùng `OpenSSL` đã build (Pi khởi build lại ~1h), một `OpenSSL` một nguồn sự thật; chạy qua `setenv` như mọi binary khác — nhất quán `no-rpath`. - Template `conf` là *bản đã sống sót qua test* + 3 quyết định đo: `ssl_session_cache off + ssl_session_tickets off` (ép FULL handshake — TLS 1.3 resumption bỏ qua truyền cert, số đo vô nghĩa nếu quên), `daemon off` (script cấm PID), groups qua `ssl_ecdh_curve` — tài liệu module `ngx_http_ssl` xác nhận đây là “list of curves supported by the server”, issue `nginx#532` xác nhận đường PQ, ticket **#2542**: directive này bị *bỏ qua lạng lẽ* ở server block `nginx-default` → mọi thứ nằm trong block default. **Kiểm sống (mục 9/N4):** build trọn trong sandbox → `nginx -V: built with OpenSSL 3.6.2 7 Apr 2026; nginx -t nhận X25519MLKEM768:MLKEM768:X25519` và **từ chối FAKEGROUP123** với lỗi đúng API `SSL_CTX_set1_curves_list(...)` failed; handshake TLS 1.3 sống + HTTP “ok”.

3. Công kiểm: `scripts/verify_env.sh`

Nhiệm vụ kép: (1) **gate** chống lỗi logic nguy hiểm nhất — âm thầm dùng OpenSSL hệ thống không có ML-KEM/ML-DSA (crypto fail im lặng là fail tệ nhất); (2) **bằng chứng** trả lời nguyên văn §7.3 “Document compiler versions, flags, and CPU governor settings” → `docs/env_report_<arch>.txt` mỗi máy. **Nguồn:** governor đọc từ `sysfs cpufreq` (tài liệu kernel `admin-guide/pm/cpufreq`). **Kiểm sống:** cả hai nhánh chạy thật — OpenSSL 3.0 → FAIL kèm hướng dẫn; môi trường đủ → PASS + report.

4. Nhóm đo WP2/WP5

`Makefile + src/bench_evp.cpp`

Nhiệm vụ: harness microbenchmark — một binary, thuật toán chọn bằng đối số (`bench_evp mlkem 768`), giao diện kiểu `openssl speed`. **Quyết định & nguồn:** - **EVP API** (không gọi thuật toán trực tiếp): cùng một mặt cắt đo cho cả RSA/ECC lẫn PQC — so sánh công bằng; tên thuật toán ML-KEM-768, ML-DSA-65 đúng manpage `EVP_PKEY-ML-KEM/EVP_PKEY-ML-DSA` (`docs.openssl.org 3.5`: “added in OpenSSL 3.5”). - Đồng hồ `CLOCK_MONOTONIC_RAW` — **nguyên văn đề §8.3**; cycles đọc kèm để cross-check (tỉ lệ thời gian ↔ tỉ lệ chu kỳ phải khớp — và đã khớp trên số thật, mục 9/N1). - **Tham số hóa qua env** (`BENCH_ITERS=2000`, `BENCH_KEYGEN_ITERS=50`, `BENCH_WARMUP=20`, `BENCH_CSV=path` — dòng 182-184, 604): tham số vị trí dành cho thuật toán, cấu hình đi đường env để xuyên qua script bao ngoài không sửa code — `BENCH_ITERS=5000` make bench lan vào mọi lượt gọi. Warm-up bị loại đúng §7.5; keygen ít vòng vì RSA keygen ~10ms/cái. - Báo median/mean/stddev/CI — đúng bộ §7.4; `BENCH_CSV` xuất mẫu thô cho bootstrap CI tuần 11. - Makefile: dò `lib/lib64`, `-pthread` (cách hiện đại, thay `-lpthread`), `SSLROOT` tự đọc từ `versions.env`, `$(error)` khi thiếu (fail to be loud); target thuần cộng `tls/analyze/tlsmini` về sau — tôn trọng luật “chỉ thêm không sửa”. **Kiểm:** compile 0 cảnh báo; chạy thật trên LOQ các phiên trước; K-batch ghi trong `KICH_BAN (5 × make bench → data/raw/<arch>/summary_batch$i.csv)`.

`scripts/run_micro.sh`

Nhiệm vụ: quét 12 thuật toán → `data/summary_micro_<arch>.csv` (long format `algo,metric,value`) + raw per-algo. Long format là chủ ý: `analyze.py` pivot tự do, thêm metric không vỡ schema.

`scripts/measure_memory.sh`

Nhiệm vụ: peak RSS per-algorithm → `memory_<arch>.csv`. **Nguồn:** GNU `time -v` đọc “Maximum resident set size” — đúng công cụ để §7.4 chỉ định (“peak RSS via `/usr/bin/time -v`”); nhấn mạnh `/usr/bin/time` vì builtin của shell không có RSS. Run ngắn đủ — footprint là *đỉnh*, không phải throughput.

`scripts/measure_codesize.sh`

Nhiệm vụ: text/data/bss/total của thư viện → `codesize_<arch>.csv`. **Nguồn:** `binutils size` — đúng để §7.4 (“size utility, `readelf -S`”). **Caveat ghi thẳng:** `libcrypto/liboqs` là số *trộn*; per-algorithm chỉ có qua `.a` PQClean (`CODESIZE_LIBS=...`) — `clean` vs `avx2/aarch64` = “tốc độ đổi bao nhiêu byte” cho Analysis.

5. Nhóm WP3: `scripts/run_liboqs_speed.sh`

Nhiệm vụ: chạy `speed_kem/speed_sig` trên **cả hai cây** × 6 thuật toán → raw evidence + `liboqs_speed_<arch>.csv` — biến WP3 từ “in màn hình” thành “dữ liệu phân tích được” (Artifacts

§11.4 đòi raw CSVs). **Quyết định & nguồn:** - Dùng đúng bộ công cụ mà chính OQS từng dùng trong hạ tầng profiling của họ (README repo open-quantum-safe/profiling, mục Purpose: “speed_sig and speed_kem... test_sig_mem and test_kem_mem”) — repo đó nay deprecated, ta cite làm *tài liệu phương pháp*, không dùng code. - Tự thích nghi: cây chưa build → skip kèm thông báo (x86 chỉ ref vẫn ra CSV; ARM cần đủ ref+opt — checklist bắt đúng lỗ hổng này). - Parser awk lọc theo cấu trúc bảng (≥ 7 cột, cột 2 là số) — **viết ngược từ output thật** của bạn, không từ tưởng tượng. **Kiểm sống (mục 9/N3):** parser chạy trên chính output 10/6 của bạn → 3 dòng CSV đúng từng chữ số (30.985/30.963/36.250).

6. Nhóm WP4 — ba tầng tích hợp TLS

Thiết kế trục đôi của WP4 dựa trên một sự thật giao thức (xác nhận bởi V. Dukhovni, maintainer OpenSSL, trên openssl-users): **KEM là trao đổi khóa ephemeral, không nằm trong certificate** → trục *chứng thư* (chữ ký: RSA/ECDSA/ML-DSA) độc lập với trục *group* (KEM: X25519/hybrid/MLKEM768). Repo học thuật pq-tls-benchmark chia *kex/* và *sig/* theo đúng ranh giới này — hai thiết kế độc lập hội tụ.

scripts/gen_tls_certs.sh

Quyết định & nguồn: tên ML-DSA-65 nguyên văn manpage EVP_PKEY-ML-DSA; sinh khóa **hai bước** genpkey → req -x509 -key (chỉ dùng cú pháp được tài liệu hóa chắc chắn, né rủi ro -newkey với tên alg mới); khóa ra ~/pqc/tls/ — **ngoài repo**, .gitignore chặn thêm một lớp; idempotent + FORCE, CERT_SET mở rộng ma trận.

scripts/bench_tls.sh (phương án A — s_server, cô lập mật mã)

Quyết định & nguồn: group X25519 / X25519MLKEM768 / MLKEM768 đúng manpage SSL_CTX_set1_groups_list 3.5 (hybrid chính thức: X25519MLKEM768, SecP256r1MLKEM768, SecP384r1MLKEM1024; “DEFAULT list selects X25519MLKEM768 as one of the predicted keyshares”); latency = vòng s_client tuần tự (median/mean/p95, §7.5), throughput = **bảo s_client song song** (CONC worker × DUR giây). **Vì sao không wrk:** wrk của distro link OpenSSL hệ thống → không có ML-KEM; wrk chỉ được phép làm *đối chứng công cụ* trên hàng X25519. Hàng fail ghi NA thay vì chết — dữ liệu trung thực hơn dữ liệu đẹp. Cột cert_bytes = proxy network-overhead §9 (bytes thật trên đây: lệnh tshark trong KICH_BAN mục 1.2).

scripts/bench_tls_nginx.sh + template conf (phương án B — server sản xuất)

Quyết định & nguồn: cùng *phương pháp đo* với bench_tls.sh (chủ ý — hai bộ số đối chứng nhau); thêm trục WORKERS_LIST="1 auto" trả lời nguyên văn §7.4 “single-threaded and multi-threaded servers” — thử s_server không làm được; CSV thêm cột server, workers, tên file tls_handshake_nginx-<arch>.csv để analyze.py tự nhặt **không sửa một dòng** code phân tích; vòng đời server: sed template → -p rundir → daemon off → PID trực tiếp → readiness probe bằng chính one_handshake.

src/tls_mini_server.c (phương án C — server tự viết, §7.6 mức sâu nhất)

Quyết định & nguồn: - Ba tầng ba chủ: TCP của ta (socket/bind/listen/accept — bài NT106), TLS 1.3 **cấu hình** qua libssl (SSL_CTX_set_min_proto_version(TLS1_3_VERSION) — không tự chế giao thức: định luật đừng-tự-viết-crypto), HTTP tối thiểu của ta (đọc tới \r\n\r\n, trả 200 + Connection: close → mỗi kết nối = một full handshake = một đơn vị đo sạch). - Mẫu theo **demo chính chủ** demos/guide/tls-server-block.c + demos/sslecho (đã curl xác minh pattern). - **Đồng hồ phía server** quanh SSL_accept, CLOCK_MONOTONIC_RAW — cùng đồng hồ với bench_evpt và s_timer.c của Paquin-Stebila-Tamvada (đảo phía client→server); mỗi kết nối in conn, tls, group, handshake_us — cột group (SSL_get_negotiated_group/SSL_group_to_name) là **bằng chứng cứng** phiên đo thật sự chạy MLKEM768. - TLS_GROUPS qua env, **fail loudly** khi OpenSSL từ chối — soi gương

hành vi nginx; không set thì dùng default thư viện (3.5+ đã ưu tiên X25519MLKEM768). **Kiểm sống (mục 9/N6)**: 3 kết nối HTTPS liên tiếp OK; client ép TLS 1.2 bị alert protocol version (alert 70) — min-version có rắng; TLS_GROUPS=FAKEGROUP123 → FATAL kèm lỗi gốc OpenSSL; warm-up lộ trong chính số đo: conn#1 7996μs vs #2-3 ~3800μs.

7. Tổng hợp & trình diễn

scripts/analyze.py

Nhiệm vụ: gom mọi CSV → analysis_out/tables.md + PNG — đúng danh sách chart §9 (“latency vs parameter set, throughput vs concurrency, bytes vs algorithm”) + bảng Cross-platform ratio (bắc cầu hai máy **chỉ bằng tỉ lệ**) + bảng speedup ref/opt (WP3). **Quyết định**: chỉ stdlib csv + matplotlib (pandas không bắt buộc — bớt phụ thuộc, §16 vẫn thỏa); matplotlib văng → vẫn ra bảng (degrade gracefully); đọc schema *thật* của từng CSV; nhấn arch suy từ tên file. **Kiểm sống**: chạy thật end-to-end với fixture đúng schema — **bắt bug thật** (split (“_”) [-1] cắt “x86_64” thành “64” → removeprefix, mục 9/N7); khối WP3 tái tạo đúng bảng $\times 2.89/\times 2.75/\times 2.64$ từ số thật.

scripts/demo.sh

Deliverable 5 nguyên văn (“short recording... benchmark runs and TLS handshake comparisons”) — 3 màn ~2 phút: chứng minh môi trường → micro sống (ML-KEM-768 vs RSA-2048) → so handshake 3 group trên cert ML-DSA (best-of-3). Thiết kế cho quay màn hình: mỗi màn một banner, không cần tham số.

8. Tài liệu & báo cáo

- README.md — cửa chính: Quickstart hai nền tảng + **Bảng chọn phương án** + bảng nghiệm thu mỗi-sản-phẩm-một-lệnh + ánh xạ repo ↔ §17.
 - docs/KICH_BAN_TRIEN_KHAI.md — kịch bản đo: thứ tự bắt buộc/tùy chọn từng máy, K-batch, nghỉ-nguội Pi (/sys/class/thermal), checklist, mục nginx + tls_mini.
 - docs/report/main.tex — khung XeLaTeX tiếng Việt đủ mục, \maybefig tự ẩn hình chưa sinh (biên dịch được từ ngày đầu), **9 nguồn đều kiểm sống** (vượt mức ≥ 6 §6), Limitations viết sẵn (energy/MCU/PMU/QEMU).
 - docs/WP1_*.md/pdf — tra cứu 12 file môi trường + giải thích x86-vs-ARM.
-

9. Nhật ký bằng chứng thực nghiệm

Mỗi cơ chế trong repo đều đã chạy thật ít nhất một lần. Đây là sổ lab của dự án — “tâm huyết” ở dạng kiểm chứng được.

#	Ngày	Thí nghiệm	Kết quả then chốt
N1	10/06	speed_kem ML-KEM-768 hai cây trên LOQ (số của chính sinh viên)	ref 30.985/30.963/36.250 μ s vs opt 10.705/11.255/13.718 μ s → × 2.89 /× 2.75 /× 2.64 ; cross-check chu kỳ CPU khớp; banner ref="SSE SSE2"+AES:OpenSSL vs opt="AVX2..." +AES:NI
N2	10/06	Cross-compile PQClean aarch64 trên x86	FAIL thật tại __asm_NTT.o (assembly tay không cross) → sinh guard chéo-kiến-trúc trong fetch_pqclean.sh; test lại: chặn đúng kèm hướng dẫn
N3	11/06	Parser run_liboqs_speed trên output thật N1	3 dòng CSV đúng từng chữ số; analyze.py tái tạo đúng bảng speedup từ số thật
N4	11/06	Build nginx 1.30.2 + OpenSSL 3.6.2 trong sandbox, test trọn	nginx -V: built with OpenSSL 3.6.2 7 Apr 2026; nginx -t nhận groups PQC, từ chối FAKEGROUP123 (SSL_CTX_set1_curves_list failed); handshake TLS1.3 sống + HTTP ok; return cần rewrite module → template chuyển file tĩnh
N5	11/06	Sự cố LD_LIBRARY_PATH trong N4	openssl hệ thống + libcrypto của ta → chết vì thiếu openssl.cnf (bản sandbox đi tắt bỏ install_ssldirs) — mình chúng sống cho install_ssldirs trong build_openssl.sh thật
N6	11/06	tls_mini_server compile + chạy sống	make tlsmini 0 cảnh báo; 3×HTTPS ok; TLS1.2 bị alert 70; TLS_GROUPS giả → FATAL; CSV server-side lộ warm-up 7996 μ s → 3832/3773 μ s

#	Ngày	Thí nghiệm	Kết quả then chốt
N7	11/06	Smoke-test analyze.py fixture đúng schema	Bắt bug split ("_") vs "x86_64" → removeprefix; 9 PNG + tables.md đủ 5 khối
N8	11/06	Kiểm tiền lệ học thuật	Clone xvzcf/pq- tls-benchmark: s_timer.c dùng SSL_connect+CLOCK_MONOTONIC server = nginx → kiến trúc WP4 hội tụ độc lập với ePrint 2019/1447; demos OpenSSL (sslecho, guide) curl xác minh
N9	11/06	Cross-configure liboqs aarch64 (toolchain repo)	CMake nhận GNU 13.3.0 aarch64, "Configuring done", 0 cảnh báo
N10	11/06	Cổng QA cuối toàn repo	19/19 script bash -n sạch; py_compile OK; 6/6 target Makefile parse; toolchain cmake -P sạch; report XeLaTeX 0 lỗi 0 thiếu glyph

10. Đối chiếu đề ↔ repo (bản vĩnh viễn)

Mục đề	Hiện thân
§7.1 ma trận thuật toán	bench_evp đủ rsa/ecdsa/ecdh/mlkem/mldsa + bảng iso-security (Cat1/3/5; RSA-2048 baseline ~112-bit)
§7.2 thư viện	build_liboqs (ref/opt) · fetch_pqclean (+deprecation) · build_oqsprovider (kế nhiệm fork)
§7.3 tái lập	versions.env · Dockerfile · docker_buildx · cross_build+toolchain · verify_env → docs/*.commit + env_report
§7.4 harness & metrics	bench_evp+run_micro (wall+cycles, median/CI) · bench_tls/bench_tls_nginx (latency+throughput, 1 auto workers) · measure_memory (GNU time -v) · measure_codesize (size)
§7.5 thống kê	warm-up loại trừ · K-batch · paired ref↔opt cùng máy · bắc cầu bằng tỉ lệ
§7.6 tích hợp "tự làm"	tls_mini_server.c (TCP+HTTP tự viết, TLS1.3 chuẩn tài liệu, đo SSL_accept, in group) + nginx
§8.3 flow 6 bước	khớp 1-1 Quickstart; CLOCK_MONOTONIC_RAW nguyên văn
§9 + §11.4	analyze.py → analysis_out (được commit) + data/raw/
§11.2/.5	docs/report/main.tex (9 nguồn kiểm sống) · scripts/demo.sh

Mục đề	Hiện thân
§13	energy (thiếu Monsoon/INA) + MCU/pqm4: khai phạm vi trong Limitations + KICH_BAN §0
§17	bảng ánh xạ trong README (energy: out-of-scope có hồ sơ)